CJAVA

siempre para apoyarte

Visión

Poder aportar al desarrollo del País usando tecnología Java.

Edwin Maraví
emaravi@cjavaperu.com

CJAVA

Spring MVC

Edwin Maraví

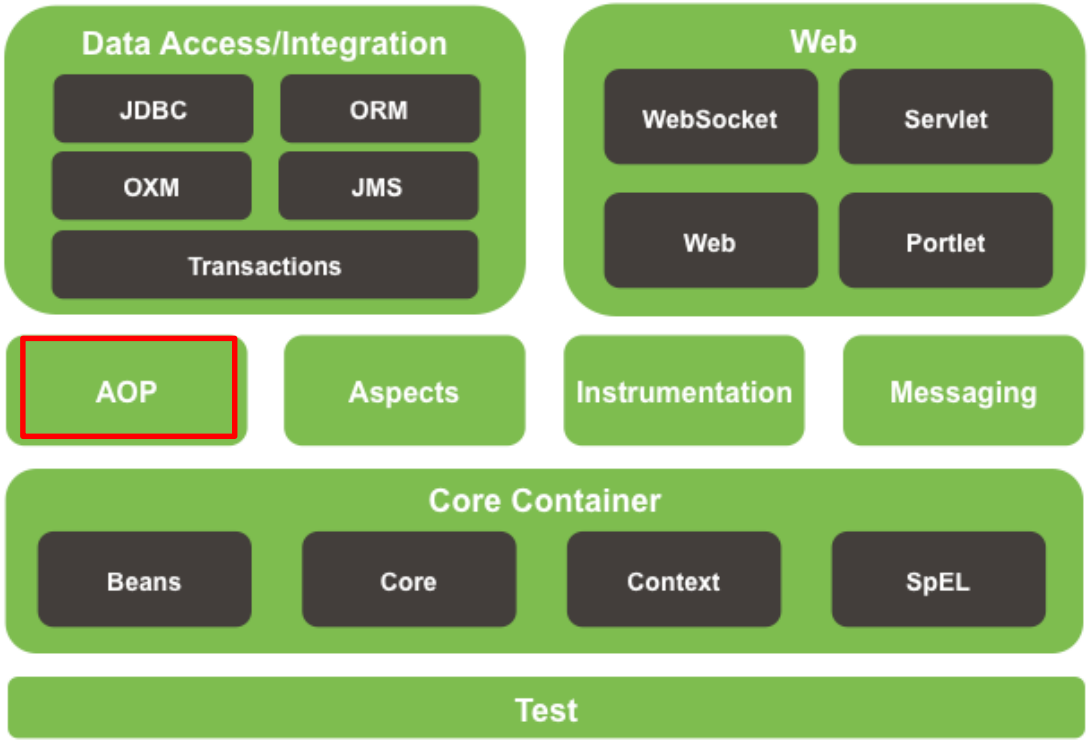
emaravi@cjavaperu.com

CJAVA
siempre para apoyarte

Objetivos

- Aprender conceptos básicos de Spring AOP

Spring AOP


Data Access/Integration
JDBC
ORM
OXM
JMS
Transactions
Web
WebSocket
Servlet
Web
Portlet
AOP
Aspects
Instrumentation
Messaging
Core Container
Beans
Core
Context
SpEL
Test

Evolución del Software

1° Código Spaghetti:

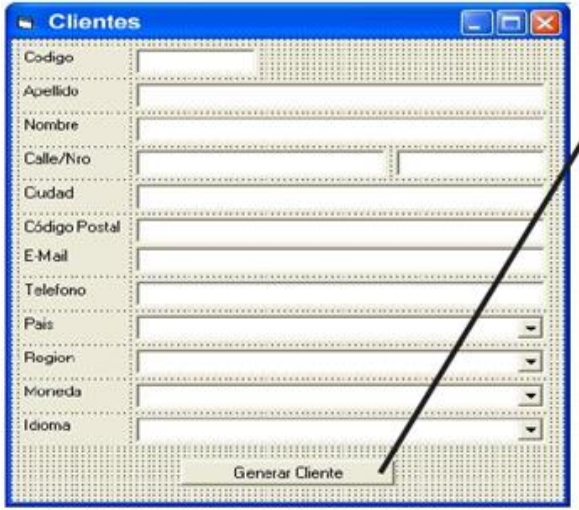
No existía un orden al programar, datos y funcionalidad se mezclaban sin una línea que lo dividiera de forma clara y correcta. En conclusión, era difícil dar mantenimiento al programa cuando este crecía, no se podía reutilizar código, el entendimiento del código demandaba mucho esfuerzo.

```
#include <iostream> // biblioteca para uso de cout
using namespace std;

int main()
{
   int numero;
   int factor;
   char resp[12];

   do {
      cout << "Introduce un número entero: ";
      cin >> numero;
      factor = 2;
      while(numero >= factor*factor) {
         if(!(numero % factor)) {
            cout << factor << " * ";
            numero = numero / factor;
```

Evolución del Software

2° y 3° Programación Estructura (Descomposición Funcional):

Se estructuro agrupando en funciones.




```
Sub grabarCliente(Codigo As String,
                  Apellido As String,
                  Nombre As String,
                  Calle As String,
                  Ciudad As String)

'Codigo para grabar Clientes'

End Sub
```

¿Que ocurría si mas adelante, se decide que el cliente no debe registrar Ciudad ni País, y SÍ un campo llamado contacto?

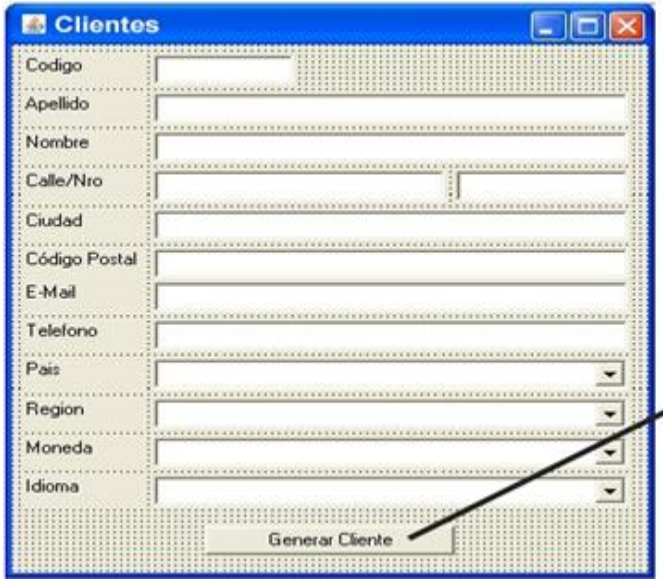
Problema: Se hubiera tenido que modificar los parámetros en la función grabarCliente y en cualquier otra función que grabe, consulte o edite un cliente.

Solución: Encapsular estas propiedades en una estructura llamada Cliente. ¿¿¿Estaremos hablando de **Clases y Objetos**???

Evolución del Software

4° Programación Orientada a Objetos:

Permite encapsular funcionalidades en clases.



cliente
-codigo:String -apellido:String -nombre:String
+getNombre():String

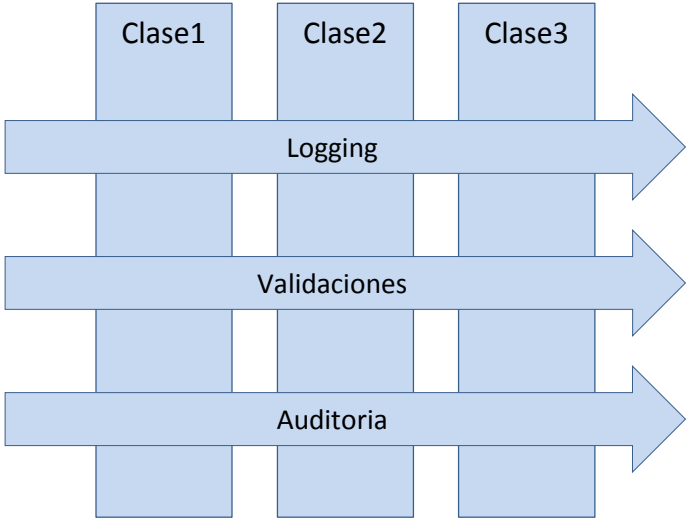
```
public void grabarCliente(Cliente objCliente){
      //Codigo para grabar Clientes
}
```

Evolución del Software

5° Descomposición de Aspectos:

Permite desacoplar funcionalidades del sistema, y trabajarlas de manera transversal.

Ejemplo: la auditoría del sistema, la seguridad, la transaccionabilidad, etc.



Problema

Supongamos que el método **grabarCliente** debe tener las siguientes funcionalidades:

1. Verifica si el usuario en sesión tiene o no permiso para grabar clientes. Si pasa la validación, entonces:
2. Inicia la transacción con la base de datos.
3. Graba el nuevo cliente en tabla.
4. Graba en la tabla de auditoria que un nuevo registro ha sido añadido a la tabla cliente, indicando cual es el usuario responsable de la operación.
5. Hacer commit o rollback a la transacción, dependiendo de si fue exitosa o fallida la operación.

```java
public class ClienteServiceImpl {
    public void grabarCliente(Cliente cliente, Usuario beanUser) {
        if(SecurityUtil.validatePermitSave(beanUser)==false){
            return;
        }
        try{
            con.setAutoCommit(false);
            clienteDAO.grabarCliente(cliente);  //EJECUTA INSERT EN TABLA CUSTOMER
            auditoriaDAO.grabarAuditoria(auditoria); //EJECUTA INSERT EN TABLA AUDIT
            con.commit();
        }catch(Exception ex){
            con.rollback();
        }
    }
}
```

ClienteServiceImpl.java

Problema

Partiendo de la premisa anterior:

Si pensamos en separación de aspectos, llegamos a la conclusión de que:

1)Aspecto: Seguridad

Aspecto: Auditoria 4)

Supongamos que el método **grabarCliente** debe tener las siguientes funcionalidades:

1) Verifica si el usuario en sesión tiene o no permiso para generar clientes. Si pasa la validación, entonces:
2) Inicia la transacción con la base de datos.
3) Graba el nuevo cliente en tabla.
4) Graba en la tabla de auditoria que un nuevo registro ha sido añadido a la tabla cliente, indicando cual es el usuario responsable de la operación.
5) Hacer commit o rollback a la transacción, dependiendo de si fue exitosa o fallida la operación.

Objetivo Principal

Aspecto: Transaccionalidad 2) y 5)

Problema

Partiendo de la premisa anterior en el código:

Si pensamos en separación de aspectos, llegamos a la conclusión de que:

1)Aspecto: Seguridad

Aspecto: Auditoria 4)

```
public class ClienteServiceImpl {
    public void grabarCliente(Cliente cliente, Usuario beanUser) {
        if(SecurityUtil.validatePermitSave(beanUser)==false){
            return;
        }
        try{
            con.setAutoCommit(false);
            clienteDAO.grabarCliente(cliente);   //EJECUTA INSERT EN TABLA CUSTOMER
            auditoriaDAO.grabarAuditoria(auditoria); //EJECUTA INSERT EN TABLA AUDIT
            con.commit();
        }catch(Exception ex){
            con.rollback();
        }
    }
}
```

Objetivo Principal

Aspecto: Transaccionalidad 2) y 5)

Solucion

Partiendo de la premisa anterior en el código:

Así debería ser nuestro código, optimizado con AOP:

Aspecto: Auditoria 4)

1)Aspecto: Seguridad

```
public class ClienteServiceImpl {
    public void grabarCliente(Cliente cliente, Usuario beanUser) {
        clienteDAO.grabarCliente(cliente);  //EJECUTA INSERT EN TABLA CUSTOMER
    }
}
```

Sólo debemos preocuparnos en el objetivo principal, que es grabar un registro de cliente y lo demás separarlo como aspectos

Aspecto: Transaccionalidad 2) y 5)

CJAVA
siempre para apoyarte

Que es AOP?

- Paradigma de programación cuya intención es permitir una adecuada modularización de las aplicaciones y permitir una mejor separación de componentes.
- Es un complemento para la programación orientada a objetos (OOP).
- Por medio de AOP podemos modificar dinámicamente un modelo estático a fin de incluir funcionalidades secundarias (transaccionabilidad, seguridad, auditoria, etc).

CJAVA
siempre para apoyarte

Términos de AOP

- **Aspecto (Aspect):** es una funcionalidad o característica transversar (cross-cutting) que vamos a separar. Por ejemplo la auditoria.

- **Punto de Corte (PointCut):** Define donde se aplicara el aspecto. Se especifica mediante expresiones regulares o mediante patrones de nombres (clases, métodos o campos)

- **Consejo (Advice):** Es la implementación del aspecto y será insertada en el punto de corte especificado. Se especifica el momento de inserción:

  Before, After returning, After throwing, After y Around

CJAVA
siempre para apoyarte

Términos de AOP

AspectJ

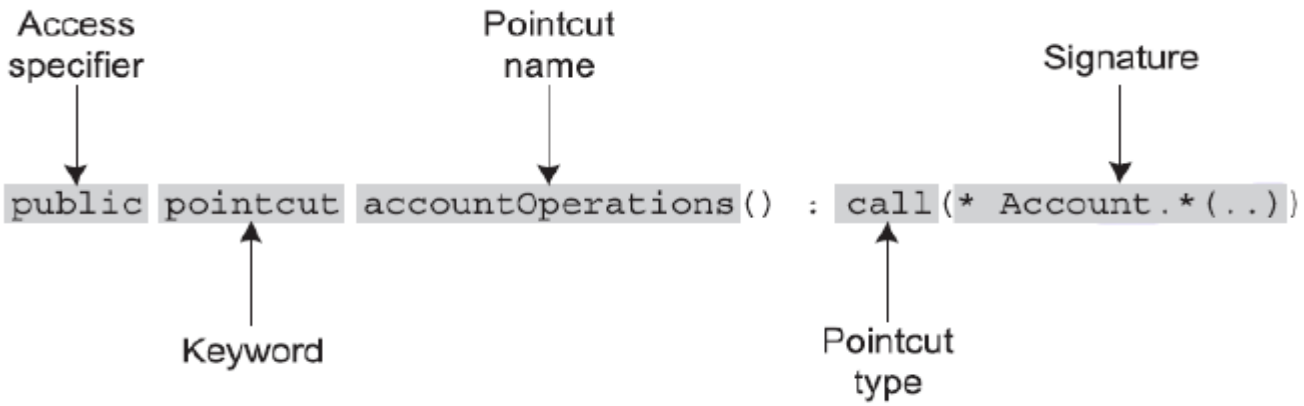
Es un lenguaje de programación orientado por aspectos construido como una extensión al lenguaje java.

Spring AOP

Es un modulo de Spring Framework que te permite realizar programación orientado a objetos. Soporta AspectJ y se puede utilizar de manera mas sencilla.

Sintaxis de AspectJ

PointCut



Advice

```
before() : accountOperations(){
    System.out.println("...");
}
```

Spring AOP

Spring AOP presenta una estructura simplificada para el desarrollo y utilización de AOP:

Por XML:

```
<?xml version="1.0"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:context="http://www.springfra
    xmlns:aop="http://www.springframework.org/schema/aop"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
                        http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
                        http://www.springframework.org/schema/context
                        http://www.springframework.org/schema/context/spring-context-4.0.xsd
                        http://www.springframework.org/schema/aop
                        http://www.springframework.org/schema/aop/spring-aop-4.0.xsd">

    <aop:aspectj-autoproxy />

    <bean id="loggingAspectoXml" class="com.cjavaperu.xml.aspecto.LoggingAspectoXml" />

    <bean id="personaDaoImpl" class="com.cjavaperu.dao.PersonaDaoImpl" />

    <aop:config>

        <aop:aspect id="aspectLoggging" ref="loggingAspectoXml">

            <aop:pointcut id="pointCutBefore"
                expression="execution(* com.cjavaperu.dao.PersonaDaoImpl.insertar(..))" />
            <aop:before method="logBefore" pointcut-ref="pointCutBefore" />
```

Spring AOP

Por Anotaciones:

```
@Configuration
@ComponentScan({"com.cjavaperu.anotacion", "com.cjavaperu.dao"})
@EnableAspectJAutoProxy
public class AppConfig {

}
```

```
@Component
@Aspect
public class LoggingAspecto {

    @Before("execution(* com.cjavaperu.dao.PersonaDaoImpl.insertar(..))")
    public void logBefore(JoinPoint joinPoint) {

        System.out.println("logBefore() esta ejecutandose!");
        System.out.println("logBefore() método : "
                + joinPoint.getSignature().getName());
    }
```

Spring AOP

Manejo de Transacciones con AOP

Spring ofrece elementos XML para manejo de transacciones de forma declarativa. Estos elementos se encuentran en el espacio de nombre **tx** y pueden utilizarse añadiendo el esquema **spring-txt** a su archivo XML de configuración de Spring:

```xml
<?xml version="1.0"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
                        http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
                        http://www.springframework.org/schema/context
                        http://www.springframework.org/schema/context/spring-context-4.0.xsd
                        http://www.springframework.org/schema/tx
                        http://www.springframework.org/schema/tx/spring-tx-4.0.xsd
                        http://www.springframework.org/schema/aop
                        http://www.springframework.org/schema/aop/spring-aop-4.0.xsd">
```

Spring AOP

Manejo de Transacciones con AOP

El espacio de nombre **tx** ofrece varios elementos nuevos de configuración XML, pero el que vamos a usar es el **<tx:advice>**. Ejemplo:

```
<tx:advice id="txAdvice" transaction-manager="txManager">
    <tx:attributes>
        <tx:method name="buscar*" read-only="true" propagation="SUPPORTS" />
        <tx:method name="guardar*" read-only="false" propagation="REQUIRED"
            isolation="DEFAULT" rollback-for="Throwable" />
    </tx:attributes>
</tx:advice>
```

Las seis facetas de las transacciones se especifican en los atributos del elemento **<tx:method>**

Atributo	Propósito
isolation	Especifica el nivel de aislamiento de la transacción
no-rollback-for	Especifica las excepciones para las cuales la transacción deberá continuar y no ser eliminada.
propagation	Define la regla de propagación de la transacción
read-only	Especifica que una transacción es de sólo lectura.
rollback-for	Especifica las excepciones comprobadas por las que una transacción debe ser eliminada y no consignada
timeout	Define un intervalo para una transacción de larga duración.

Spring AOP

Manejo de Transacciones con AOP

<tx:advice>: Solo define una notificación AOP para notificar métodos con limites de transacción, pero solo es una notificación de transacción, no un aspecto transaccional completo. En ningún sitio se indica qué bean debe ser notificado; para eso necesitamos un punto de corte. Para definir completamente el aspecto, debemos definir un notificador. Aquí es donde esta implicado el espacio de nombre **aop**.

```
<aop:config>
    <aop:advisor pointcut="execution(* com.cjavaperu.service.*Service*.*(..))"
        advice-ref="txAdvice" />
</aop:config>
```

```
<aop:config>
    <aop:pointcut id="serviceOperation"
        expression="execution(* com.cjavaperu.service.*Service*.*(..))" />
    <aop:advisor pointcut-ref="serviceOperation" advice-ref="txAdvice" />
</aop:config>
```

El **pointcut** utiliza una expresión de punto de corte **AspectJ** para indicar que este notificador deberá notificar a todos los método de las clases que contengan el nombre Service en el paquete service.

Links de Referencia

Web Spring

- http://spring.io

Spring Framework

- http://projects.spring.io/spring-framework

Spring Framework Data Access

- http://docs.spring.io/spring/docs/current/spring-framework-reference/html/spring-data-tier.html

CJAVA

siempre para apoyarte

Gracias